Advanced Features Testing and Validation Guide

개요

이 가이드는 AutoNAT, DCUtR, Relay를 통합한 고급 P2P 애플리케이션의 테스트 및 검증 방법을 제공합니다. 단위 테스트부터 통합 테스트, 성능 테스트, 그리고 실제 네트워크 환경에서의 검증까지 포괄적으로 다룹니다.

테스트 전략

1. 테스트 피라미드

```
┌─────────────────────────────────────┐
| E2E Tests | ← 실제 네트워크 환경
| (End-to-End) |
├─────────────────────────────────────┤
| Integration Tests | ← 다중 노드 시나리오
| (Component Integration) |
├─────────────────────────────────────┤
| Unit Tests | ← 개별 컴포넌트
| (Individual Components) |
└─────────────────────────────────────┘
```

2. 테스트 환경 구성

```rust
// test_environment.rs
use libp2p::{PeerId, Multiaddr};
use std::{collections::HashMap, time::Duration};
use tokio::sync::mpsc;

/// 테스트 환경 설정
#[derive(Debug, Clone)]
pub  struct TestEnvironment {
pub relay_servers: Vec<TestRelayServer>,
pub test_nodes: Vec<TestNode>,
pub network_conditions: NetworkConditions,
pub test_duration: Duration,
}

#[derive(Debug, Clone)]
pub  struct TestRelayServer {
pub peer_id: PeerId,
pub address: Multiaddr,
pub capacity: RelayCapacity,
}
```

```rust
#[derive(Debug, Clone)]
pub struct TestNode {
pub peer_id: PeerId,
pub node_type: TestNodeType,
pub nat_type: NATType,
pub expected_behavior: ExpectedBehavior,
}

#[derive(Debug, Clone)]
pub enum TestNodeType {
Client,
Server,
Hybrid,
}

#[derive(Debug, Clone)]
pub enum NATType {
None, // 공개 IP
FullCone, // 전체 원뿔 NAT
RestrictedCone, // 제한된 원뿔 NAT
PortRestricted, // 포트 제한 NAT
Symmetric, // 대칭 NAT (홀 펀칭 어려움)
}
```

```rust
#[derive(Debug, Clone)]
pub  struct NetworkConditions {
pub latency: Duration,
pub packet_loss: f64,
pub bandwidth_limit: Option<u64>,
pub connection_drops: f64,
}

#[derive(Debug, Clone)]
pub  struct ExpectedBehavior {
pub should_connect_directly: bool,
pub should_use_relay: bool,
pub should_hole_punch: bool,
pub max_connection_time: Duration,
}

#[derive(Debug, Clone)]
pub  struct RelayCapacity {
pub max_reservations: usize,
pub max_circuits: usize,
pub max_bandwidth: u64,
}
```

```rust
/// 테스트 환경 빌더
pub struct TestEnvironmentBuilder {
relay_servers: Vec<TestRelayServer>,
test_nodes: Vec<TestNode>,
network_conditions: NetworkConditions,
test_duration: Duration,
}

impl TestEnvironmentBuilder {
pub fn new() -> Self {
Self {
relay_servers: Vec::new(),
test_nodes: Vec::new(),
network_conditions: NetworkConditions {
latency: Duration::from_millis(50),
packet_loss: 0.0,
bandwidth_limit: None,
connection_drops: 0.0,
},
test_duration: Duration::from_secs(300), // 5분
}
}
```

```
pub  fn add_relay_server(mut  self, capacity: RelayCapacity) -> Self {

let relay_server = TestRelayServer {

peer_id: PeerId::random(),

address: format!("/ip4/127.0.0.1/tcp/{}", 4000 + self.relay_servers.len())

.parse()

.unwrap(),

capacity,

};

self.relay_servers.push(relay_server);

self

}



pub  fn add_test_node(mut  self, node_type: TestNodeType, nat_type: NATType) ->

let expected_behavior = match (&node_type, &nat_type) {

(_, NATType::None) => ExpectedBehavior {

should_connect_directly: true,

should_use_relay: false,

should_hole_punch: false,

max_connection_time: Duration::from_secs(5),

},

(_, NATType::Symmetric) => ExpectedBehavior {

should_connect_directly: false,
```

```
should_use_relay: true,
should_hole_punch: false,
max_connection_time: Duration::from_secs(15),
},
_ => ExpectedBehavior {
should_connect_directly: false,
should_use_relay: true,
should_hole_punch: true,
max_connection_time: Duration::from_secs(10),
},
};

let test_node = TestNode {
peer_id: PeerId::random(),
node_type,
nat_type,
expected_behavior,
};
self.test_nodes.push(test_node);
self
}

pub  fn with_network_conditions(mut  self, conditions: NetworkConditions) -> Sel
```

```
self.network_conditions = conditions;
self
}

pub fn with_test_duration(mut self, duration: Duration) -> Self {
self.test_duration = duration;
self
}

pub fn build(self) -> TestEnvironment {
TestEnvironment {
relay_servers: self.relay_servers,
test_nodes: self.test_nodes,
network_conditions: self.network_conditions,
test_duration: self.test_duration,
}
}
}
```

단위 테스트

1. AutoNAT 테스트

```rust
// tests/autonat_tests.rs

use libp2p::autonat;

use std::time::Duration;


#[tokio::test]

async  fn test_autonat_status_detection() {

let config = autonat::Config {

timeout: Duration::from_secs(5),

boot_delay: Duration::from_secs(1),

retry_interval: Duration::from_secs(10),

confidence_max: 2,

only_global_ips: false, // 테스트 환경

..Default::default()

};


let local_key = libp2p::identity::Keypair::generate_ed25519();

let  mut autonat_behaviour = autonat::Behaviour::new(

local_key.public().to_peer_id(),

config,

);


// 테스트 서버 추가
```

```
let server_peer_id = PeerId::random();

let server_address: Multiaddr = "/ip4/127.0.0.1/tcp/4001".parse().unwrap();

autonat_behaviour.add_server(server_peer_id, Some(server_address));


// 초기 상태 확인

assert_eq!(autonat_behaviour.nat_status(), &autonat::NatStatus::Unknown);

assert_eq!(autonat_behaviour.confidence(), 0);

}


#[tokio::test]

async  fn test_autonat_server_throttling() {

let config = autonat::Config {

throttle_clients_global_max: 2,

throttle_clients_peer_max: 1,

throttle_clients_period: Duration::from_secs(1),

..Default::default()

};


let local_key = libp2p::identity::Keypair::generate_ed25519();

let autonat_behaviour = autonat::Behaviour::new(

local_key.public().to_peer_id(),

config,

);
```

```
// 스로틀링 테스트 로직
// 실제 구현에서는 내부 상태에 접근하여 스로틀링 동작 확인
}

#[tokio::test]
async fn test_autonat_confidence_system() {
// 신뢰도 시스템 테스트
// 연속된 성공/실패에 따른 신뢰도 변화 확인
}
```

2. DCUtR 테스트

```
// tests/dcutr_tests.rs
use libp2p::dcutr;

#[tokio::test]
async fn test_dcutr_hole_punching_success() {
// 성공적인 홀 펀칭 시나리오 테스트
let local_key = libp2p::identity::Keypair::generate_ed25519();
let dcutr_behaviour = dcutr::Behaviour::new(local_key.public().to_peer_id());

// 홀 펀칭 성공 시나리오 시뮬레이션
```

```
}

#[tokio::test]
async fn test_dcutr_hole_punching_failure() {
// 홀 펀칭 실패 시나리오 테스트
// 최대 재시도 횟수 확인
}

#[tokio::test]
async fn test_dcutr_address_candidates() {
// 주소 후보 관리 테스트
// LRU 캐시 동작 확인
}
```

3. Relay 테스트

```
// tests/relay_tests.rs
use libp2p::relay;
use std::time::Duration;

#[tokio::test]
async fn test_relay_reservation() {
let config = relay::Config {
```

```
max_reservations: 10,
max_reservations_per_peer: 2,
reservation_duration: Duration::from_secs(60),
..Default::default()
};

let local_key = libp2p::identity::Keypair::generate_ed25519();
let relay_behaviour = relay::Behaviour::new(
local_key.public().to_peer_id(),
config,
);

// 예약 요청 처리 테스트
}

#[tokio::test]
async fn test_relay_circuit_establishment() {
// 회로 설정 테스트
}

#[tokio::test]
async fn test_relay_rate_limiting() {
// Rate limiting 테스트
```

```
}
```

통합 테스트

1. 다중 노드 연결 테스트

```
// tests/integration_tests.rs
use  crate::advanced_features_integration::*;
use std::time::Duration;
use tokio::time::timeout;

#[tokio::test]
async  fn test_multi_node_connection() {
tracing_subscriber::fmt()
.with_env_filter("debug")
.init();

// 테스트 환경 설정
let test_env = TestEnvironmentBuilder::new()
.add_relay_server(RelayCapacity {
max_reservations: 100,
max_circuits: 50,
max_bandwidth: 10_000_000, // 10MB/s
```

```
})
.add_test_node(TestNodeType::Client, NATType::FullCone)
.add_test_node(TestNodeType::Client, NATType::RestrictedCone)
.add_test_node(TestNodeType::Server, NATType::None)
.with_test_duration(Duration::from_secs(60))
.build();

// 릴레이 서버 시작
let  mut relay_server = create_relay_server(&test_env.relay_servers[0]).await.un
let relay_task = tokio::spawn(async  move {
relay_server.run().await
});

// 클라이언트 노드들 시작
let  mut client_nodes = Vec::new();
for test_node in &test_env.test_nodes {
let node = create_test_node(test_node, &test_env.relay_servers).await.unwrap();
client_nodes.push(node);
}

// 연결 테스트 실행
let connection_results = test_node_connections(&mut client_nodes).await;
```

```
// 결과 검증
for (i, result) in connection_results.iter().enumerate() {
let expected = &test_env.test_nodes[i].expected_behavior;
match result {
Ok(connection_info) => {
assert!(connection_info.connection_time <= expected.max_connection_time,
"Connection took too long: {:?} > {:?}",
connection_info.connection_time, expected.max_connection_time);
if expected.should_connect_directly {
assert!(connection_info.is_direct, "Expected direct connection");
}
if expected.should_use_relay {
assert!(connection_info.used_relay, "Expected relay usage");
}
if expected.should_hole_punch {
assert!(connection_info.hole_punched, "Expected hole punching");
}
}
Err(e) => {
panic!("Connection failed for node {}: {}", i, e);
}
}
}
```

```
// 정리
relay_task.abort();
}

async fn create_relay_server(relay_config: &TestRelayServer) -> Result<Advanced
let config = NodeConfig {
enable_relay_server: true,
relay_servers: vec![],
..Default::default()
};

let mut node = AdvancedP2PNode::new(config).await?;
node.start().await?;
Ok(node)
}

async fn create_test_node(test_config: &TestNode, relay_servers: &[TestRelaySer
let relay_list: Vec<(PeerId, Multiaddr)> = relay_servers
.iter()
.map(|r| (r.peer_id, r.address.clone()))
.collect();
```

```
let config = NodeConfig {
enable_relay_server: false,
relay_servers: relay_list,
aggressive_hole_punching: true,
..Default::default()
};

let  mut node = AdvancedP2PNode::new(config).await?;
node.start().await?;
Ok(node)
}

#[derive(Debug)]
struct ConnectionInfo {
connection_time: Duration,
is_direct: bool,
used_relay: bool,
hole_punched: bool,
}

async  fn test_node_connections(nodes: &mut [AdvancedP2PNode]) -> Vec<Result<Con
let  mut results = Vec::new();
// 모든 노드 쌍에 대해 연결 테스트
```

```
for i in  0..nodes.len() {
for j in (i + 1)..nodes.len() {
let start_time = std::time::Instant::now();
let target_peer_id = *nodes[j].swarm.local_peer_id();
match timeout(
Duration::from_secs(30),
nodes[i].connect_to_peer(target_peer_id, vec![])
).await {
Ok(Ok(_)) => {
let connection_time = start_time.elapsed();
let connection_info = ConnectionInfo {
connection_time,
is_direct: nodes[i].is_directly_connected(&target_peer_id),
used_relay: nodes[i].is_relay_connected(&target_peer_id),
hole_punched: connection_time < Duration::from_secs(10) &&
nodes[i].is_directly_connected(&target_peer_id),
};
results.push(Ok(connection_info));
}
Ok(Err(e)) => {
results.push(Err(e));
}
Err(_) => {
```

```
results.push(Err("Connection timeout".into()));
}
}
}
}
results
}
```

2. NAT 시나리오 테스트

```
// tests/nat_scenarios.rs

#[tokio::test]
async  fn test_symmetric_nat_scenario() {
// 대칭 NAT 환경에서의 연결 테스트
// 홀 펀칭이 실패하고 릴레이 연결만 성공해야 함
let test_env = TestEnvironmentBuilder::new()
.add_relay_server(RelayCapacity::default())
.add_test_node(TestNodeType::Client, NATType::Symmetric)
.add_test_node(TestNodeType::Client, NATType::Symmetric)
.build();

// 테스트 실행
```

```
let results = run_nat_test(&test_env).await;
// 대칭 NAT에서는 홀 펀칭이 실패해야 함
for result in results {
assert!(result.used_relay, "Should use relay for symmetric NAT");
assert!(!result.hole_punched, "Hole punching should fail for symmetric NAT");
}
}

#[tokio::test]
async  fn test_full_cone_nat_scenario() {
// 전체 원뿔 NAT 환경에서의 연결 테스트
// 홀 펀칭이 성공해야 함
let test_env = TestEnvironmentBuilder::new()
.add_relay_server(RelayCapacity::default())
.add_test_node(TestNodeType::Client, NATType::FullCone)
.add_test_node(TestNodeType::Client, NATType::FullCone)
.build();

let results = run_nat_test(&test_env).await;
// 전체 원뿔 NAT에서는 홀 펀칭이 성공해야 함
for result in results {
assert!(result.hole_punched, "Hole punching should succeed for full cone NAT");
assert!(result.is_direct, "Should establish direct connection");
```

```
}

}


async fn run_nat_test(test_env: &TestEnvironment) -> Vec<ConnectionInfo> {

// NAT 테스트 실행 로직

vec![]

}
```

성능 테스트

1. 처리량 테스트

```
// tests/performance_tests.rs

use std::sync::atomic::{AtomicU64, Ordering};

use std::sync::Arc;

use tokio::time::{Duration, Instant};


#[tokio::test]

async fn test_relay_throughput() {

let message_count = Arc::new(AtomicU64::new(0));

let bytes_transferred = Arc::new(AtomicU64::new(0));

let start_time = Instant::now();

let test_duration = Duration::from_secs(30);
```

```rust
// 릴레이 서버 설정
let mut relay_server = create_high_capacity_relay_server().await.unwrap();
// 클라이언트 노드들 설정
let mut sender = create_test_client().await.unwrap();
let mut receiver = create_test_client().await.unwrap();


// 연결 설정
let receiver_peer_id = *receiver.swarm.local_peer_id();
sender.connect_to_peer(receiver_peer_id, vec![]).await.unwrap();


// 메시지 전송 태스크
let message_count_clone = Arc::clone(&message_count);
let bytes_transferred_clone = Arc::clone(&bytes_transferred);
let send_task = tokio::spawn(async move {
let message = vec![0u8; 1024]; // 1KB 메시지
while start_time.elapsed() < test_duration {
// 메시지 전송 (실제 구현에서는 커스텀 프로토콜 사용)
message_count_clone.fetch_add(1, Ordering::Relaxed);
bytes_transferred_clone.fetch_add(message.len() as u64, Ordering::Relaxed);
tokio::time::sleep(Duration::from_millis(1)).await;
}
});
```

```rust
// 테스트 실행
tokio::select! {
_ = send_task => {},
_ = tokio::time::sleep(test_duration) => {},
}

// 결과 분석
let total_messages = message_count.load(Ordering::Relaxed);
let total_bytes = bytes_transferred.load(Ordering::Relaxed);
let elapsed = start_time.elapsed();

let messages_per_second = total_messages as f64 / elapsed.as_secs_f64();
let bytes_per_second = total_bytes as f64 / elapsed.as_secs_f64();
let mbps = bytes_per_second / (1024.0 * 1024.0);

println!("Performance Results:");
println!(" Messages: {} ({:.2} msg/s)", total_messages, messages_per_second);
println!(" Throughput: {:.2} MB/s", mbps);
println!(" Duration: {:?}", elapsed);

// 성능 기준 검증
assert!(messages_per_second > 100.0, "Message rate too low: {:.2}", messages_per
```

```
assert!(mbps > 1.0, "Throughput too low: {:.2} MB/s", mbps);
}

async fn create_high_capacity_relay_server() -> Result<AdvancedP2PNode, Box<dyn
let config = NodeConfig {
enable_relay_server: true,
relay_servers: vec![],
autonat_config: autonat::Config {
max_peer_addresses: 50,
throttle_clients_global_max: 1000,
throttle_clients_peer_max: 10,
..Default::default()
},
..Default::default()
};

let mut node = AdvancedP2PNode::new(config).await?;
node.start().await?;
Ok(node)
}

async fn create_test_client() -> Result<AdvancedP2PNode, Box<dyn std::error::Er
let config = NodeConfig {
```

```
enable_relay_server: false,
aggressive_hole_punching: false, // 성능 테스트에서는 릴레이 사용
..Default::default()
};

let mut node = AdvancedP2PNode::new(config).await?;
node.start().await?;
Ok(node)
}
```

2. 지연시간 테스트

```
#[tokio::test]
async fn test_connection_latency() {
let mut latency_measurements = Vec::new();
let test_iterations = 100;

for i in 0..test_iterations {
let start_time = Instant::now();
// 새로운 연결 설정
let mut client1 = create_test_client().await.unwrap();
let mut client2 = create_test_client().await.unwrap();
```

```
let client2_peer_id = *client2.swarm.local_peer_id();

// 연결 시도

match client1.connect_to_peer(client2_peer_id, vec![]).await {

Ok(_) => {

let latency = start_time.elapsed();

latency_measurements.push(latency);

println!("Connection {} established in {:?}", i, latency);

}

Err(e) => {

eprintln!("Connection {} failed: {}", i, e);

}

}

// 정리

tokio::time::sleep(Duration::from_millis(100)).await;

}



// 통계 계산

if !latency_measurements.is_empty() {

let avg_latency = latency_measurements.iter().sum::<Duration>() / latency_measur

let min_latency = latency_measurements.iter().min().unwrap();

let max_latency = latency_measurements.iter().max().unwrap();

println!("Latency Statistics:");

println!(" Average: {:?}", avg_latency);
```

```
println!(" Min: {:?}", min_latency);

println!(" Max: {:?}", max_latency);

println!(" Success rate: {:.2}%",

latency_measurements.len() as f64 / test_iterations as f64 * 100.0);


// 성능 기준 검증

assert!(avg_latency < Duration::from_secs(10), "Average latency too high: {:?}",

assert!(latency_measurements.len() as f64 / test_iterations as f64 > 0.8,

"Success rate too low");

}

}
```

3. 확장성 테스트

```
#[tokio::test]

async  fn test_scalability() {

let node_counts = vec![10, 50, 100, 200];

for node_count in node_counts {

println!("Testing with {} nodes", node_count);

let start_time = Instant::now();

// 릴레이 서버 생성

let  mut relay_server = create_high_capacity_relay_server().await.unwrap();

let relay_peer_id = *relay_server.swarm.local_peer_id();
```

```
let relay_address: Multiaddr = "/ip4/127.0.0.1/tcp/4001".parse().unwrap();

// 다수의 클라이언트 노드 생성

let  mut nodes = Vec::new();

for i in  0..node_count {

let config = NodeConfig {

enable_relay_server: false,

relay_servers: vec![(relay_peer_id, relay_address.clone())],

..Default::default()

};

let  mut node = AdvancedP2PNode::new(config).await.unwrap();

node.start().await.unwrap();

nodes.push(node);

if i % 10 == 0 {

println!("Created {} nodes", i + 1);

}

}

let setup_time = start_time.elapsed();

println!("Setup time for {} nodes: {:?}", node_count, setup_time);

// 연결 테스트

let connection_start = Instant::now();

let  mut successful_connections = 0;

// 각 노드가 릴레이 서버에 연결

for (i, node) in nodes.iter_mut().enumerate() {
```

```
match node.connect_to_peer(relay_peer_id, vec![relay_address.clone()]).await {
Ok(_) => {
successful_connections += 1;
}
Err(e) => {
eprintln!("Node {} failed to connect: {}", i, e);
}
}
}
let connection_time = connection_start.elapsed();
let success_rate = successful_connections as f64 / node_count as f64;
println!("Results for {} nodes:", node_count);
println!(" Setup time: {:?}", setup_time);
println!(" Connection time: {:?}", connection_time);
println!(" Success rate: {:.2}%", success_rate * 100.0);
println!(" Connections per second: {:.2}",
successful_connections as f64 / connection_time.as_secs_f64());
// 성능 기준 검증
assert!(success_rate > 0.9, "Success rate too low for {} nodes: {:.2}%",
node_count, success_rate * 100.0);
assert!(setup_time < Duration::from_secs(node_count as u64 / 2),
"Setup time too long for {} nodes: {:?}", node_count, setup_time);
println!("✅ Scalability test passed for {} nodes\n", node_count);
```

```
}

}
```

실제 네트워크 테스트

1. 네트워크 시뮬레이션

```
// tests/network_simulation.rs

use std::collections::HashMap;

use tokio::sync::mpsc;


/// 네트워크 시뮬레이터

pub struct NetworkSimulator {

nodes: HashMap<PeerId, SimulatedNode>,

network_conditions: NetworkConditions,

message_router: MessageRouter,

}


pub struct SimulatedNode {

peer_id: PeerId,

node_type: TestNodeType,

nat_type: NATType,

connections: Vec<PeerId>,
```

```rust
message_sender: mpsc::UnboundedSender<SimulatedMessage>,
}

pub struct MessageRouter {
routes: HashMap<(PeerId, PeerId), Route>,
conditions: NetworkConditions,
}

#[derive(Debug, Clone)]
pub struct Route {
pub latency: Duration,
pub packet_loss: f64,
pub bandwidth: u64,
pub path: Vec<PeerId>, // 릴레이 경로
}

#[derive(Debug, Clone)]
pub enum SimulatedMessage {
ConnectionRequest(PeerId),
ConnectionResponse(PeerId, bool),
DataTransfer(Vec<u8>),
KeepAlive,
}
```

```rust
impl NetworkSimulator {
pub fn new(conditions: NetworkConditions) -> Self {
Self {
nodes: HashMap::new(),
network_conditions: conditions,
message_router: MessageRouter::new(conditions),
}
}

pub async fn add_node(&mut self, node_type: TestNodeType, nat_type: NATType)
let peer_id = PeerId::random();
let (tx, rx) = mpsc::unbounded_channel();
let node = SimulatedNode {
peer_id,
node_type,
nat_type,
connections: Vec::new(),
message_sender: tx,
};
self.nodes.insert(peer_id, node);
// 노드 메시지 처리 태스크 시작
self.start_node_task(peer_id, rx).await;
```

```
peer_id
}

async fn start_node_task(&self, peer_id: PeerId, mut message_receiver: mpsc::Un
tokio::spawn(async move {
while let Some(message) = message_receiver.recv().await {
match message {
SimulatedMessage::ConnectionRequest(from_peer) => {
// 연결 요청 처리
println!("Node {} received connection request from {}", peer_id, from_peer);
}
SimulatedMessage::DataTransfer(data) => {
// 데이터 전송 처리
println!("Node {} received {} bytes", peer_id, data.len());
}
_ => {}
}
}
});
}

pub async fn simulate_connection(&mut self, from: PeerId, to: PeerId) -> Resu
let from_node = self.nodes.get(&from).ok_or("From node not found")?;
```

```
let to_node = self.nodes.get(&to).ok_or("To node not found")?;


// NAT 타입에 따른 연결 가능성 확인
let can_connect_directly = self.can_connect_directly(&from_node.nat_type, &to_no
if can_connect_directly {
// 직접 연결
let route = Route {
latency: self.network_conditions.latency,
packet_loss: self.network_conditions.packet_loss,
bandwidth: self.network_conditions.bandwidth_limit.unwrap_or(u64::MAX),
path: vec![from, to],
};
Ok(route)
} else {
// 릴레이를 통한 연결
let relay_peer = self.find_relay_node().ok_or("No relay available")?;
let route = Route {
latency: self.network_conditions.latency * 2, // 릴레이로 인한 추가 지연
packet_loss: self.network_conditions.packet_loss * 1.5,
bandwidth: self.network_conditions.bandwidth_limit.unwrap_or(u64::MAX) / 2,
path: vec![from, relay_peer, to],
};
Ok(route)
```

```
}
}

fn can_connect_directly(&self, nat1: &NATType, nat2: &NATType) -> bool {
match (nat1, nat2) {
(NATType::None, _) | (_, NATType::None) => true,
(NATType::FullCone, NATType::FullCone) => true,
(NATType::RestrictedCone, NATType::RestrictedCone) => true,
(NATType::PortRestricted, NATType::PortRestricted) => false,
(NATType::Symmetric, _) | (_, NATType::Symmetric) => false,
_ => false,
}
}

fn find_relay_node(&self) -> Option<PeerId> {
self.nodes
.iter()
.find(|(_, node)| matches!(node.node_type, TestNodeType::Server))
.map(|(peer_id, _)| *peer_id)
}

pub  async  fn run_simulation(&mut  self, duration: Duration) -> SimulationResul
let start_time = Instant::now();
```

```
let  mut results = SimulationResults::new();


println!("Starting network simulation for {:?}", duration);


// 시뮬레이션 실행
while start_time.elapsed() < duration {
// 주기적으로 연결 시도
self.attempt_random_connections(&mut results).await;
// 네트워크 조건 변경 시뮬레이션
self.simulate_network_changes().await;
tokio::time::sleep(Duration::from_secs(1)).await;
}


results.total_duration = start_time.elapsed();
results
}


async  fn attempt_random_connections(&mut  self, results: &mut SimulationResults
let node_ids: Vec<PeerId> = self.nodes.keys().copied().collect();
if node_ids.len() < 2 {
return;
}
```

```
// 랜덤하게 두 노드 선택
let from = node_ids[rand::random::<usize>() % node_ids.len()];
let to = node_ids[rand::random::<usize>() % node_ids.len()];
if from != to {
let start_time = Instant::now();
match  self.simulate_connection(from, to).await {
Ok(route) => {
let connection_time = start_time.elapsed();
results.successful_connections += 1;
results.total_connection_time += connection_time;
if route.path.len() == 2 {
results.direct_connections += 1;
} else {
results.relayed_connections += 1;
}
}
Err(_) => {
results.failed_connections += 1;
}
}
}
}
```

```
async fn simulate_network_changes(&mut self) {
// 네트워크 조건 변경 시뮬레이션
if rand::random::<f64>() < 0.1 { // 10% 확률로 조건 변경
self.network_conditions.latency = Duration::from_millis(
50 + rand::random::<u64>() % 200
);
self.network_conditions.packet_loss = rand::random::<f64>() * 0.05; // 0-5%
}
}
}

impl MessageRouter {
fn new(conditions: NetworkConditions) -> Self {
Self {
routes: HashMap::new(),
conditions,
}
}
}

#[derive(Debug, Default)]
pub struct SimulationResults {
pub successful_connections: u64,
```

```
pub failed_connections: u64,

pub direct_connections: u64,

pub relayed_connections: u64,

pub total_connection_time: Duration,

pub total_duration: Duration,

}


impl SimulationResults {

fn new() -> Self {

Default::default()

}


pub  fn print_summary(&self) {

let total_attempts = self.successful_connections + self.failed_connections;

let success_rate = if total_attempts > 0 {

self.successful_connections as f64 / total_attempts as f64 * 100.0

} else {

0.0

};


let avg_connection_time = if  self.successful_connections > 0 {

self.total_connection_time / self.successful_connections as u32

} else {
```

```
Duration::ZERO
};

println!("\n=== Simulation Results ===");
println!("Duration: {:?}", self.total_duration);
println!("Total connection attempts: {}", total_attempts);
println!("Successful connections: {}", self.successful_connections);
println!("Failed connections: {}", self.failed_connections);
println!("Success rate: {:.2}%", success_rate);
println!("Direct connections: {}", self.direct_connections);
println!("Relayed connections: {}", self.relayed_connections);
println!("Average connection time: {:?}", avg_connection_time);
}
}

#[tokio::test]
async  fn test_network_simulation() {
let conditions = NetworkConditions {
latency: Duration::from_millis(100),
packet_loss: 0.02, // 2%
bandwidth_limit: Some(10_000_000), // 10 MB/s
connection_drops: 0.01, // 1%
};
```

```rust
let  mut simulator = NetworkSimulator::new(conditions);


// 다양한 노드 타입 추가
simulator.add_node(TestNodeType::Server, NATType::None).await; // 릴레이 서버
simulator.add_node(TestNodeType::Client, NATType::FullCone).await;
simulator.add_node(TestNodeType::Client, NATType::RestrictedCone).await;
simulator.add_node(TestNodeType::Client, NATType::Symmetric).await;
simulator.add_node(TestNodeType::Client, NATType::PortRestricted).await;


// 시뮬레이션 실행
let results = simulator.run_simulation(Duration::from_secs(60)).await;
// 결과 출력 및 검증
results.print_summary();
assert!(results.successful_connections > 0, "No successful connections");
let success_rate = results.successful_connections as f64 /
(results.successful_connections + results.failed_connections) as f64;
assert!(success_rate > 0.7, "Success rate too low: {:.2}%", success_rate * 100.0
}
```

테스트 자동화

1. CI/CD 통합

```yaml
# .github/workflows/test.yml
name: Advanced P2P Tests

on:
push:
branches: [ main, develop ]
pull_request:
branches: [ main ]

jobs:
unit-tests:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
- uses: actions-rs/toolchain@v1
with:
toolchain: stable
- name: Run unit tests
run: cargo test --lib

integration-tests:
```

```yaml
runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- uses: actions-rs/toolchain@v1

with:

toolchain: stable

- name: Run integration tests

run: cargo test --test integration_tests

env:

RUST_LOG: debug



performance-tests:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- uses: actions-rs/toolchain@v1

with:

toolchain: stable

- name: Run performance tests

run: cargo test --test performance_tests --release

env:

RUST_LOG: info
```

```yaml
network-simulation:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
- uses: actions-rs/toolchain@v1
with:
toolchain: stable
- name: Run network simulation
run: cargo test --test network_simulation --release
env:
RUST_LOG: info
```

2. 테스트 리포트 생성

```rust
// tests/test_reporter.rs
use serde::{Deserialize, Serialize};
use std::fs;
use std::time::Duration;

#[derive(Debug, Serialize, Deserialize)]
pub  struct TestReport {
pub test_suite: String,
pub timestamp: String,
```

```
pub duration: Duration,
pub results: Vec<TestResult>,
pub summary: TestSummary,
}

#[derive(Debug, Serialize, Deserialize)]
pub  struct TestResult {
pub test_name: String,
pub status: TestStatus,
pub duration: Duration,
pub details: Option<String>,
pub metrics: Option<TestMetrics>,
}

#[derive(Debug, Serialize, Deserialize)]
pub  enum TestStatus {
Passed,
Failed,
Skipped,
}

#[derive(Debug, Serialize, Deserialize)]
pub  struct TestMetrics {
```

```
pub connections_per_second: Option<f64>,

pub throughput_mbps: Option<f64>,

pub success_rate: Option<f64>,

pub average_latency: Option<Duration>,

}


#[derive(Debug, Serialize, Deserialize)]

pub  struct TestSummary {

pub total_tests: usize,

pub passed: usize,

pub failed: usize,

pub skipped: usize,

pub success_rate: f64,

}


impl TestReport {

pub  fn new(test_suite: String) -> Self {

Self {

test_suite,

timestamp: chrono::Utc::now().to_rfc3339(),

duration: Duration::ZERO,

results: Vec::new(),

summary: TestSummary {
```

```
total_tests: 0,
passed: 0,
failed: 0,
skipped: 0,
success_rate: 0.0,
},
}
}

pub  fn add_result(&mut  self, result: TestResult) {
match result.status {
TestStatus::Passed => self.summary.passed += 1,
TestStatus::Failed => self.summary.failed += 1,
TestStatus::Skipped => self.summary.skipped += 1,
}
self.results.push(result);
self.summary.total_tests = self.results.len();
self.summary.success_rate = self.summary.passed as f64 / self.summary.total_test
}

pub  fn save_to_file(&self, path: &str) -> Result<(), Box<dyn std::error::Error>
let json = serde_json::to_string_pretty(self)?;
fs::write(path, json)?;
```

```
Ok(())
}

pub  fn print_summary(&self) {
println!("\n=== Test Report Summary ===");
println!("Test Suite: {}", self.test_suite);
println!("Timestamp: {}", self.timestamp);
println!("Duration: {:?}", self.duration);
println!("Total Tests: {}", self.summary.total_tests);
println!("Passed: {}", self.summary.passed);
println!("Failed: {}", self.summary.failed);
println!("Skipped: {}", self.summary.skipped);
println!("Success Rate: {:.2}%", self.summary.success_rate * 100.0);
if  self.summary.failed > 0 {
println!("\nFailed Tests:");
for result in &self.results {
if matches!(result.status, TestStatus::Failed) {
println!(" - {}: {:?}", result.test_name, result.details);
}
}
}
}
```

```
}
```

결론

이 테스트 가이드는 고급 P2P 기능 통합의 품질과 안정성을 보장하기 위한 포괄적인 테스트 전략을 제공합니다:

주요 테스트 영역

1. **단위 테스트**: 개별 컴포넌트의 정확성 검증
2. **통합 테스트**: 다중 노드 시나리오에서의 상호작용 검증
3. **성능 테스트**: 처리량, 지연시간, 확장성 측정
4. **네트워크 시뮬레이션**: 다양한 네트워크 조건에서의 동작 검증

테스트 자동화

- CI/CD 파이프라인 통합
- 자동화된 테스트 리포트 생성
- 성능 회귀 감지

실제 환경 검증

- 다양한 NAT 타입 시나리오
- 네트워크 조건 변화 대응
- 장애 상황 복구 능력

이러한 체계적인 테스트를 통해 프로덕션 환경에서 안정적으로 동작하는 고품질의 P2P 애플리케이션을 구축할 수 있습니다.